

```

string first = arr[0];
string ans = " ";

```

```

t → lcp(first, ans);
return ans;

```

```

}

```

```

void lcp(string str, string &ans){
    for (int i=0; i<str.length(); i++){
        char ch = str[i];

```

```

        if (root → childCount == 1) {
            ans.push-back(ch);
            // aage badho
            int index = ch - 'a';
            root → root → children[index];

```

```

        } else {
            break;

```

```

        }
        if (root → isTerminal) {
            break;
        }

```

Que 81

Q.) Implement a phone directory.

e.g.

input: Lo

o/p: Love, Location, Lotus
(search from prefix).

suggestion

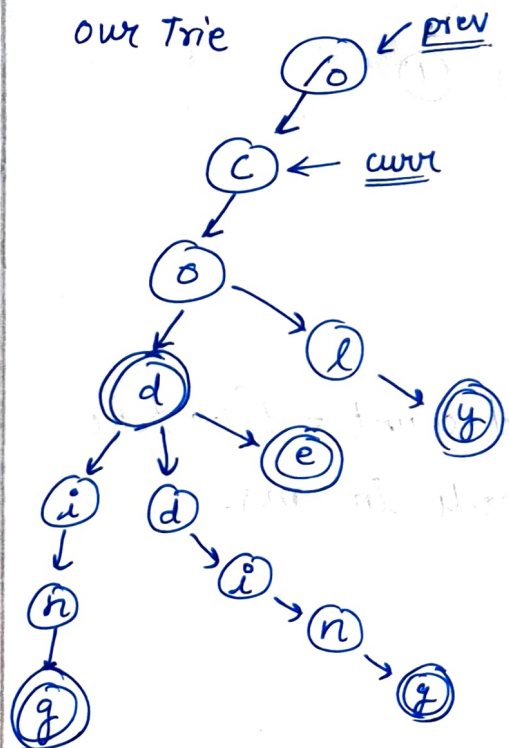
approach: (Trie)

e.g. Trie contains following words.

→ cod → code
→ coding → coly.
→ coddling

prefix = coding
(given)

our Trie



Algo:

i) read prefix str one char by one.

if (prev → next has == last char)

→ print suggestions. (recursion)

for C → all words starting from 'C'.

1st Pass over

prefixStr = C

lastChar = 0

o/p = coly, cod, coding, coddling
code.

= cod

= d.

→ cod, coding, coddling
code.

= coding

→ = coding.

ii) move prev to curr & curr to node have lastChar after printing suggestions.

code

```
vector <vector <string>> phoneDir (
vector <string> &contactList, string &
queryStr) {
```

```
Trie * t = new Trie();
```

```
// INSERT ALL CONTACT LIST IN TRIE
for (int i=0; i < contactList.size();
i++) {
```

```
string str = contactList[i];
```

```
t → insertWord(str);
}
```

```
return t → getSuggestions (queryStr);
```

```
vector <vector <string>> getSuggestions (
string str) {
```

```
Trie Node * prev = root;
```

```
vector <vector <string>> output;
```

```
string prefix = "";
```

```
for (int i=0; i < str.length(); i++) {
```

```
char lastch = str[i];
```

```
prefix.push-back (lastch);
```

```
// Check TrieNode * curr = prev →
```

```
for lchar children [
```

```
(last char) lastch - 'a' ];
```

// if not found

~~TrieNode*~~ curr = pr

if (curr == NULL) {

break;

// if found

vector<string> temp;

printSuggestions(curr, temp, prefix);

output.push_back(temp);

temp.clear();

prev = curr;

}

return output;

}

* void printSuggestions (TrieNode* curr, vector<string> &temp, string prefix) {

if (curr->isTerminal) {

temp.push_back(prefix);

~~return;~~

}

for (char ch = 'a'; ch <= 'z'; ch++) {

TrieNode* next = curr->children[ch - 'a'];

if (next != NULL) {

prefix.push_back(ch);

printSuggestions(next, temp, prefix);

```
prefix.pop-back();
```

```
} // if close
```

```
} // for loop close
```

} // f^n close.

n = length of prefix str
 m = avg length of words

T.C = $(n^2 + m^2)$

$$S.C = (n \times m) //$$

LCB2

Backtracking

Q.1) Rat in maze

- rat needs to go from $(0,0)$ to $(n-1,n-1)$ of the matrix
- rat only walks on '1'
- return path as string of direction (ULDRU...)

e.g.

	0	1	2	3
0	1	0	0	0
1	1	1	0	0
2	1	1	0	0
3	0	1	1	1

movements $\rightarrow$ U L D R

$\hookrightarrow$ solⁿ $\rightarrow$ $\left. \begin{array}{l} 1) DDRDRR \\ 2) DRD DRR \end{array} \right\}$ give all possible solⁿ(s)
in alphabetical order.

check in lexicographical order $\rightarrow$ ~~DR~~ ~~LU~~
DLRU

safe (A && B && C)

A $\rightarrow$ arr[i][j] == 1

B $\rightarrow$ visited == 0

C $\rightarrow$ ~~0~~ $0 \leq i, j \leq n-1$

} three conditions to move rat.

code

```
vector<string> searchMaze (vector<vector<int>> &arr, int n) {  
    vector<string> ans;  
    vector<vector<bool>> visited (n, vector<bool> (n, 0));  
    string path = "";  
    solve (0, 0, arr, n, visited, path);  
    return ans;  
}
```

```
void solve (int x, int y, vec <vec <int>> &arr, int n, vector <string>
&ans, vector <vector <bool>> &vis, string path) {
```

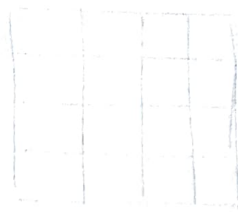
```
// base case
```

```
if (x == n-1 && y == n-1) {
    ans.push_back(path);
    return;
}
```

```
// D, L, R, U
```

```
// Down
```

```
if (isSafe (x+1, y, vis visited visited, arr, n)) {
    vis[x][y] = 1;
    solve (x+1, y, arr, n, ans, vis, path + 'D');
    vis[x][y] = 0; → BACKTRACK
}
```



```
// Left
```

```
// Right
```

```
// Up
```

```
if (isSafe (x-1, y, visited, arr, n)) {
    vis[x][y] = 1;
    solve (x-1, y, arr, n, ans, vis, path + 'U');
    vis[x][y] = 0;
}
```

$$T.C = O(4^{n^2})$$

$$S.C = O(n \times m)$$

lec 83

NOTE

Q-2) N Queen Problem

→ placing N Queens on $N \times N$ chess board such that no two queens attack each other. ($N > 3$)

N=4

	0	1	2	3
0		Q		
1				Q
2	Q			
3			Q	

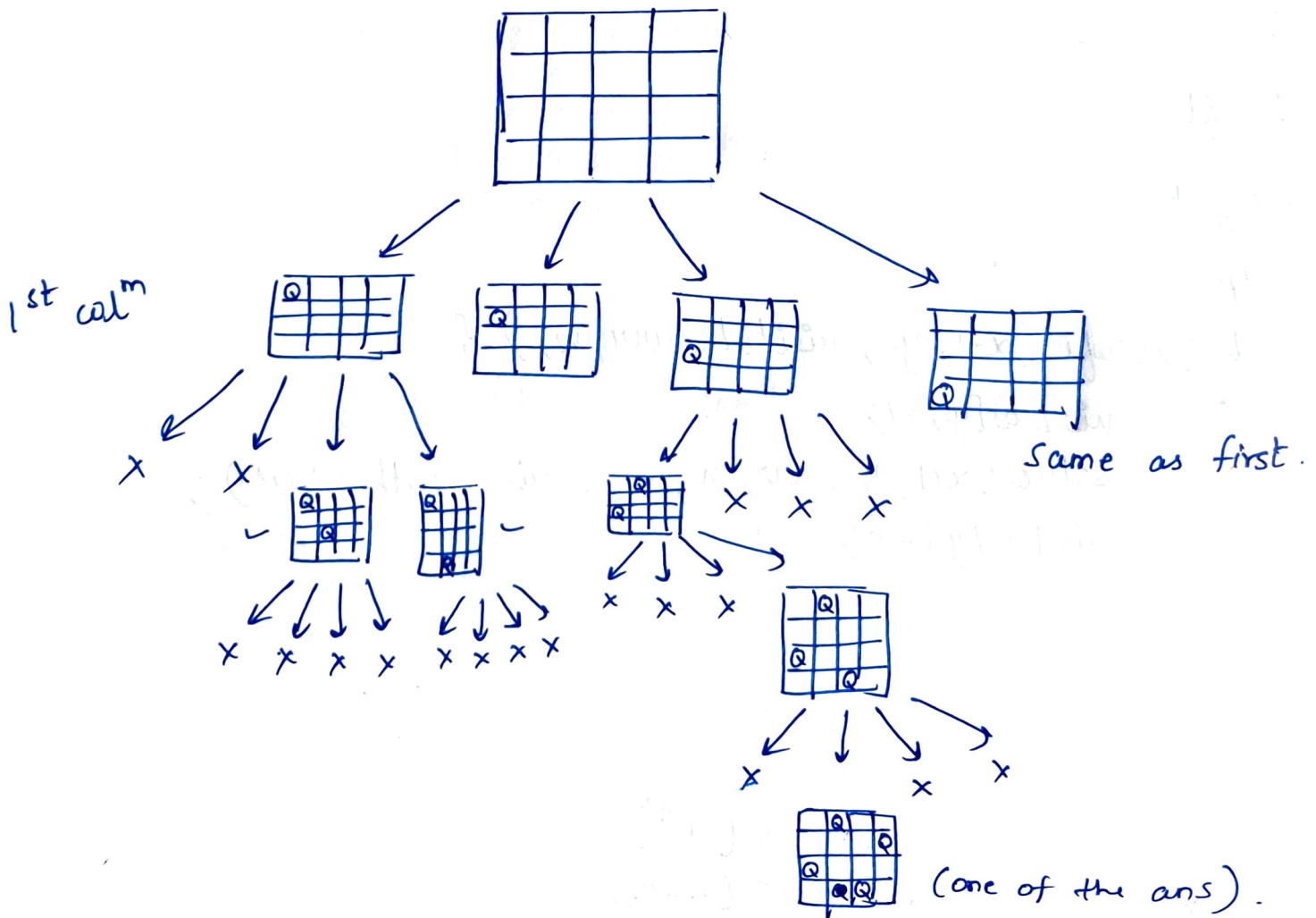
O/P = (0,1), (1,3), (2,0), (3,2)

approach :

① Place 1 Queen in each col^m such that :

i) 1 Q in 1 row

ii) 1 Q in that diagonal.



code. NOTE : WRITE FUNCTIONS IN ORDER :

```
void solve (int col, vector<vector<int>> &ans, vec.<vec<int>>
            & board, int n) {
    // base case
    if (col == n) {
        addSoln (board, ans);
        return;
    }
```

```
void addSoln (vec<vec<int>> &ans, vector<vec<int>>
              & board) {
    vector<int> temp;
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            temp.push_back (board[i][j]);
        }
    }
    ans.push_back (temp);
}
```

solve function continued. (after return/ base case).

```
for (int row=0; row<n; row++) {
    if (isSafe (row, col, board, n)) {
        board[i][j] = 1;
        solve (col+1, ans, board, n);
        board[i][j] = 0; // BACKTRACK
    }
}
```



```
bool isSafe ( int row, int col, &board , int n) {
```

```
    int x = row;
```

```
    int y = col;
```

```
    // check for same row
```

```
    while (y >= 0) {
```

```
        if (board[x][y] == 1)
```

```
    }    y--;    return false;
```

```
    x = row;
```

```
    y = col;
```

```
    // check for diagonal 1
```

```
    while (x >= 0 && y >= 0) {
```

```
        if (board[x][y] == 1)
```

```
            return false;
```

```
        y--;
```

```
        x--;
```

```
    }
```

```
    // check for diagonal 2
```

```
    x = row;
```

```
    y = col;
```

```
    while (x < n && y >= 0) {
```

```
        if (board[x][y] == 1)
```

```
            return false;
```

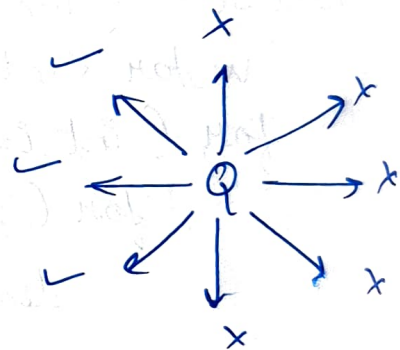
```
        y--;
```

```
        x++;
```

```
    }
```

```
    return true;
```

```
}
```



only check 3 lines
of attacks

optimisation

using map in isSafe func.

map < (n-1) + col - row >
↪ T/F

map < (col + row) > ↪ T/F

map < (row) > ↪ T/F.

↪ O(1)

Q-7) Sudoku Solver (Hard problem)

- 9×9 matrix is given (empty space $\rightarrow$ '0')
- fill '0' place with (1 to 9) digit such that each row, column & box (3×3) has its occurrence '1' only. (normal sudoku)

approach

i) Put digit in blank space one by one (from 1 to 9) and check if that digit is valid or NOT by check above 3 conditions.

i) Traverse matrix

ii) If any cell $[i][j] == 0$

↳ for (int digit = 1 to 9)

↳ if (isSafe)

↳ matrix $[i][j] = \text{val}$

↳ recursion.

↳ backtrack for other solutions.

code

```
bool isSafe (int row, int col, vector<vector<int>> &board, int val)
```

```
for (int i=0; i<board.size(); i++){
```

```
    // row check
```

```
    if (board[row][i] == val) {
```

```
        return false;
```

```
    // col check
```

```
    if (board[i][col] == val) {
```

```
        return false;
```

***** → // 3*3 matrix check

```
    if (board[3*(row/3) + i/3][3*(col/3) + i%3] == val) {
```

```
        return false;
```

```
    }
```

```
}
```

```
return true;
```

```
}
```

```
bool solve (vector<vector<int>> &board) {
```

```
    int n = board[0].size();
```

```
    for (int col=0; col<n; col++){
```

```
        for (int row=0; row<n; row++){
```

```
            // cell empty found
```

```
            if (board[row][col] == 0) {
```

```
                for (int val=1; val<=9; val++){
```

```
if (isSafe (row, col, board, val)) {
```

```
    board [row][col] = val;
```

```
    // recursive call
```

```
    bool (aageSolutionPossible = solve (board);
```

```
    if (aageSolutionPossible)
```

```
        return true;
```

```
    else {
```

```
        return false;
```

```
        board [row][col] = 0; // BACKTRACK
```

```
    } else end
```

```
    } if end
```

```
    } for end
```

```
    return false;
```

```
    } if end
```

```
    } for1 end
```

```
    } for2 end.
```

```
    return true
```

```
}
```

```
void solveSudoku (vector <vector <int>> & sudoku) {
```

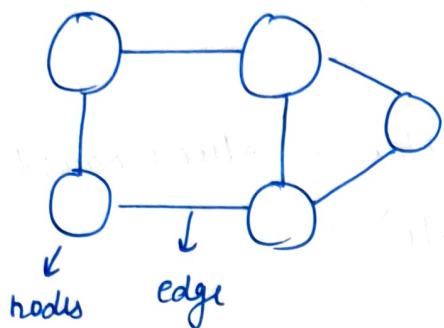
```
    solve (sudoku);
```

```
}
```

T.C = $O(9^m)$ $m \rightarrow$ total empty cells

S.C = $O(1)$.

GRAPHS



→ Data structure which is made by combination of nodes & edges.

Undirected



edges → uv
→ vu

Directed.



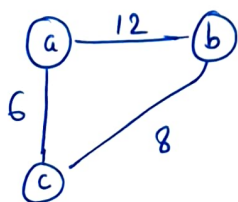
edges → uv

Node/Vertex → Entity to store data
Edge → Connects Nodes.

Degree of Node = Total Edges

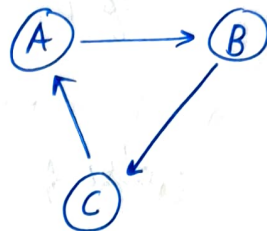
deg in directed graph } = indeg + outdeg

weighted graph



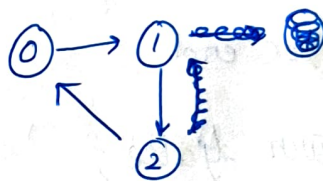
Path: Sequence of Nodes to travel from one Node to another.

Cyclic graph.



Representation of Matrix.

① Adjacency Matrix.

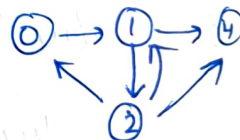


	0	1	2
0	0	1	0
1	0	0	1
2	1	0	0

S.C = $O(n^2)$
↓
no. of nodes.

② Adjacency list.

0 → 1
1 → 2, 4
2 → 0, 1, 4
4 → ϕ .



implementation by:

map < int, list <int> > ;

Implementation:

```
#include <unordered_map>
#include <list>
```

```
class graph {
```

```
public:
```

```
unordered_map <int, list
               <int>> adj;
```

```
void addEdge (int u, int v,
              bool direction) {
```

```
// dir = 0 → undirected
```

```
// dir = 1 → directed
```

```
// create an edge from u to v
```

```
adj[u].push_back(v);
```

```
if (direction == 0) {
```

```
adj[v].push_back(u);
```

```
}
```

```
}
```

```
void printAdjList () {
```

```
for (auto i: adj) {
```

```
cout << i.first << "->";
```

```
for (auto j: i.second) {
```

```
cout << j << ", ";
```

```
}
```

```
cout << endl;
```

```
}
```

```
};
```

```
int main () {
```

```
int n;
```

```
"Enter number of Nodes"
```

```
cin >> n;
```

```
int m;
```

```
"Enter edges"
```

```
cin >> m;
```

```
graph g;
```

```
for (int i=0; i<m; i++) {
```

```
int u, v;
```

```
cin >> u >> v;
```

```
g.addEdge(u, v, 0);
```

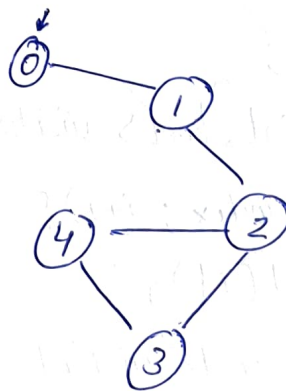
```
}
```

```
return 0;
```

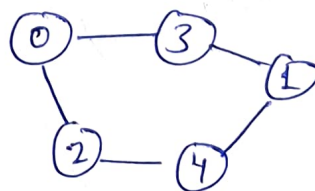
```
}
```

lec 86 Breadth First Search (BFS)

→ Traversal Technique



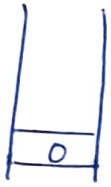
[0, 1, 2, 3, 4]



[0, 3, 2, 4, 1]

we have to track visited nodes using unordered map <node, bool>

queue (FIFO) Adj list visited



0 → 3	0 → F
1 → 2, 3, 4	1 → F
2 → 1	2 → F
3 → 0, 1	3 → F
4 → 1	4 → F

- I) frontNode (front of queue)
- II) Push in queue
- III) Mark visited and pop from queue
- IV) Print ans
- V) push all elements of frontNode from adj list which are not visited before
- VI) Repeat

~~code~~
~~vector<int> BFS (int vertex,~~
~~vector<pair<int, int>> edges) {~~
~~unordered_map~~
~~vector<int> ans;~~
~~unordered_map<int, bool> visited;~~
~~for (int i=0; i<vertex; i++) {~~
~~if (!visited[i]) {~~
~~bfs (adjlist, visited,~~
~~ans,~~

code

```
void bfs (unordered_map<int,
list<int>> &adjlist, unordered_
set
<int, bool> &visited, vector<int>
&ans, int node) {
```

```
queue<int> q;
```

```
q.push (node);
```

```
visited [node] = 1;
```

```
while (!q.empty()) {
```

```
int frontNode = q.front();
```

```
q.pop();
```

```
// store frontNode into ans
```

```
ans.push_back (frontNode);
```

```
// traverse all neighbours of
frontNode
```

```
for (auto i : adjlist [frontNode])
```

```
if (!visited[i]) {
```

```
q.push (i);
```

```
visited [i] = 1;
```

```
}
```

```
}
```

```
}
```

```
}
```



```

vector<int> BFS (int vertex, vector<pair<int, int>> edges) {
    unordered_map<int, listset<int>> adjList;
    vector<int> ans;
    unordered_map<int, bool> visited;

    prepareAdjList (adjList, edges);

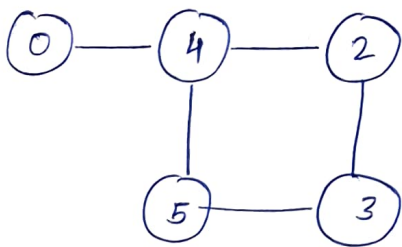
    for (int i=0; i<vertex; i++) {
        if (!visited[i]) {
            bfs (adjList, visited, ans, i);
        }
    }

    return ans;
}

```

→ for loop handles disconnected graph

lec 87 Depth First Search (DFS)



→ DFS: [0, 4, 2, 3, 5] depth first
 BFS: [0, 4, 2, 5, 3] breadth first

- ① prepare adjList
- ② go to each level (unvisited walk) using list
- ③ Print each level.

```

code void dfs (int node, unordered_map<int, bool> &visited,
    unordered_map<int, list<int>> &adj,
    vector<int> &component) {
    component.push_back (node);
    visited [node] = true;

    // recursive call for each connected node
    for (auto i: adj [node]) {
        if (!visited [i]) {
            dfs (i, visited, adj, component);
        }
    }
}

```



```
vector <vector <int>> DFS (int V,  
int E, vector <vector <int>> &edges){
```

```
// prepare adjlist
```

```
unordered_map <int, list <int>>  
adj;
```

```
for (int i=0; i < edges.size(); i++){
```

```
int u = edges[i][0];
```

```
int v = edges[i][1];
```

```
adj[u].push_back(v);
```

```
adj[v].push_back(u);
```

```
}
```

```
vector <vector <int>> ans;
```

```
unordered_map <int, bool> visited;
```

```
for (int i=0; i < V; i++){
```

```
if (!visited[i]){
```

```
vector <int> component;
```

```
dfs (i, visited, adj, component);
```

```
ans.push_back(component);
```

```
}
```

```
}
```

```
return ans;
```

```
}
```